

REPUBLIQUE DU CAMEROUN

PAIX – TRAVAIL – PATRIE

MINISTERE DE
L'ENSEIGNEMENT
SUPERIEURE



REPUBLIC OF CAMEROON

PEACE – WORK –
FATHERLAND

MINISTRY OF HIGHER
EDUCATION

FACULTY OF ENGINEERING AND TECHNOLOGY

PO BOX 63

BUEA, SOUTH WEST REGION

DEPARTMENT OF COMPUTER ENGINEERING

COURSE CODE: CEF440

COURSE TITLE: INTERNET PROGRAMMING AND MOBILE DEVELOPMENT

COURSE INSTRUCTOR: Dr. NKEMENI VALERY

TASK 4 REPORT: Software Design Document (SDD) for the Mobile-Based Attendance Management System Based on Geofencing and Facial Recognition

Presented by:

NAMES	MATRICLE
1. MEKOLLE ASHLEY ARREYMANYOR	FE22A247
2. AKO RUTH ACHERE	FE22A144
3. NGUIMENANG ZEUFACK KEREINE	FE22A266
4. TUMUTOH LYDIE-KASEY BIHNUI	FE22A321
5. TIWA DELAN NDIKA	FE22A319

MAY 2025

Table of Content

1.	Introduction.....	4
1.1.	Purpose of the Document.....	4
1.2.	Scope of the System.....	4
1.3.	Intended Audience	4
1.4.	Organization of Work.....	5
2.	System Overview	5
2.1.	System Objectives.....	5
2.2.	Key Features	6
2.3.	Users and Roles.....	7
2.4.	Technologies Used	8
3.	System Architecture	9
3.1.	Overview	9
3.2.	Architectural Design	9
3.3.	System Components.....	10
4.	System Modules.....	11
4.1.	User Management Module.....	11
4.2.	Course and Session Management Module.....	11
4.3.	Attendance Check-in Module	11
4.4.	Lecturer-Assisted Check-In Module.....	12
4.5.	Leave and Dispute Management Module	12
4.6.	Attendance Logging and History Module.....	12
4.7.	Dashboard and Reporting Module	13
4.8.	Notification Module.....	13
5.	UML Diagrams	13
5.1.	Use Case Diagram.....	15
5.1.1.	Diagram.....	15
5.1.2.	Use Case description.....	16
5.2.	Class Diagram.....	23

5.3.	Context Diagram	25
5.4.	Data Flow Diagram	27
5.5.	Sequence Diagrams	30
5.6.	Deployment Diagram	39
5.7.	Activity Diagrams	41
6.	Conclusion	50

System Design Document

1. Introduction

1.1. Purpose of the Document

The purpose of this System Design Document (SDD) is to outline a comprehensive view of the structure, design principles, and intended implementation approach for the Mobile-Based Attendance Management System. This system integrates geofencing and facial recognition to automate and validate student attendance in real-time. The SDD serves as a central reference for all project stakeholders to ensure alignment between the design and the intended functionality. It will guide the development team through the system architecture, interfaces, modules, and interaction between system components.

1.2. Scope of the System

The Attendance Management System is designed to improve the accuracy, transparency, and efficiency of tracking student attendance in academic environments. By utilizing mobile GPS services and camera-based facial recognition, the system ensures that students are physically present within predefined geofenced areas during check-in. The platform supports multiple user roles, including students, lecturers, and administrators, each with specific access rights and responsibilities. Key functionalities include real-time attendance check-in, course session management, leave request handling, attendance reporting, and role-based dashboards.

The system is implemented as a cross-platform mobile application integrated with a cloud backend (Firebase) to provide real-time database synchronization, authentication, and secure data handling. It is suitable for deployment in universities, colleges, training institutes, or corporate training environments.

1.3. Intended Audience

This document is intended to be read and used by the following stakeholders:

- **System Architects and Backend Developers:** To understand data flow, system interactions, and module boundaries.
- **Frontend Developers and UI/UX Designers:** To understand navigation flow and screen interactions.
- **Project Managers and Coordinators:** To track design milestones and system deliverables.
- **Quality Assurance Teams:** To align test cases and validation plans with the design logic.
- **Academic Institutions or Clients:** To understand how the solution meets operational requirements.

1.4. Organization of Work

This document is divided into structured chapters to present the design clearly:

- **Chapter 1:** Introduces the document purpose, scope, and stakeholders.
- **Chapter 2:** Provides a high-level overview of the system's goals, features, and user roles.
- **Chapter 3:** Outlines the overall architecture and major system components.
- **Chapter 4:** Describes key functional modules and how they interact.
- **Chapter 5:** Includes UML diagrams representing system workflows and data movement.
- **Chapter 6:** Summarizes design choices, outlines current limitations, and proposes future enhancements.

2. System Overview

2.1. System Objectives

The Mobile-Based Attendance Management System aims to revolutionize how attendance is managed in academic institutions by leveraging mobile technologies, specifically GPS-based geofencing and camera-enabled facial recognition. The objective is to create an efficient, tamper-proof, and user-friendly platform for both students and Lecturers.

Key objectives include:

- **Accurate Attendance Logging:** Ensure only students who are physically present in the classroom are marked present.
- **Identity Verification:** Prevent impersonation through biometric facial recognition.
- **Minimized Manual Effort:** Automate attendance collection and reporting to reduce administrative workload.
- **Real-Time Insights:** Enable instant access to attendance records and statistics for better decision-making.
- **Leave and Dispute Management:** Provide structured channels for students to request absences or challenge errors.
- **Mobile Accessibility:** Provide students and lecturers with a seamless mobile interface to interact with the system anytime, anywhere.

These goals are in line with the functional requirements of real-time check-in, face and location verification, course session creation, and report generation as documented in the SRS.

2.2. Key Features

The system includes a set of integrated features designed to ensure secure, accurate, and efficient attendance tracking. Below is a detailed explanation of each key feature:

2.2.1. User Registration and Authentication:

- This is the gateway to accessing the system. Students, lecturers, and admins register and log in using secure credentials via Firebase Authentication.
- Authentication includes email verification, password protection, and optional multi-factor authentication to enhance security.
- After successful login, users are redirected to role-specific dashboards.

2.2.2. Course Enrollment:

- Students can browse a list of available courses based on semester, faculty, or department.
- With a simple tap, students can request to join a course. If approved (automatically or manually), the course appears on their dashboard.
- Lecturers can view a list of enrolled students and control enrollment manually if needed.

2.2.3. Real-Time Attendance Check-In:

- Students check in during active sessions using a single button.
- The app first checks geofence status by verifying GPS coordinates.
- Next, the camera captures the student's face. Facial recognition is used to confirm identity.
- If both checks pass, the system records the check-in time and status instantly.
- This dual-validation ensures attendance is both location-specific and identity-verified.

2.2.4. Geofencing Verification:

- A virtual boundary (usually a 50m radius around the lecture hall) is created using the GPS coordinates of the classroom.
- The system uses real-time location data to validate that the student is within the boundary.
- Students attempting to check in outside this area are denied and prompted to move within the allowed range.

2.2.5. Facial Recognition:

- Uses either Firebase ML Kit or a dedicated facial recognition API.
- Captures a live image and compares it to the registered biometric profile.
- Includes liveness detection to prevent fraud using photos or videos.
- If a match score exceeds a set threshold (e.g., 95%), the student is confirmed present.

2.2.6. Lecturer-Assisted Check-In:

- Designed for sessions with multiple students or where individual check-ins may be slow.

- Lecturers activate the assisted check-in interface which allows batch scanning of multiple student faces.
- Recognized students are shown in a list for the lecturer to approve or adjust.
- Speeds up the attendance process for large classes while maintaining verification.

2.2.7. Leave Requests and Dispute Handling:

- Students can submit requests for future absences (leave) or to challenge attendance records (disputes).
- Requests include details such as date, reason, and any supporting files.
- Lecturers receive notifications and can approve, deny, or add comments.
- Students are notified of the decision and can view the result from their dashboard.

2.2.8. Attendance Dashboard:

- Each user has a personalized dashboard.
- Students see their attendance statistics per course, recent sessions, and pending requests.
- Lecturers can view attendance history for each session, course-wide analytics, and manage disputes.
- Dashboards include charts, graphs, and filter options for clarity.

2.2.9. Report Downloading:

- Lecturers and administrators can generate attendance reports filtered by course, student, or date range.
- Reports can be exported in PDF or CSV format.
- This supports institutional record-keeping, evaluation, and auditing.

These features are closely mapped to the system's functional requirements such as user management, real-time attendance check-in, biometric verification, dashboard access, and dispute handling. They also align with non-functional goals including ease of use, security, and performance.

2.3. Users and Roles

The system serves three main user groups, each with specific roles and permissions tailored to their needs.

2.3.1. Students:

- Register, log in, and join courses.
- Check in using GPS and facial recognition.
- View attendance records.
- Submit leave and dispute forms Requests.

2.3.2. Lecturers:

- Create and manage course sessions.
- Conduct student-assisted check-ins.
- Monitor and manage attendance per course.
- Review and respond to student requests.
- Generate downloadable reports.

2.3.3. Administrators:

- Manage overall system settings.
- Monitor user activities.
- Access aggregated reports across all courses and users.

The clear division of responsibilities and access control aligns with the system's non-functional requirements for security, role-based permissions, and usability.

2.4. Technologies Used

The system is built using a combination of modern, scalable, and mobile-first technologies to support its core goals of accessibility, automation, and security. These technologies were chosen based on their ability to satisfy the system's functional and non-functional requirements, such as real-time performance, ease of use, cost-efficiency, and secure data management.

2.4.1. React Native:

- A cross-platform framework for building native mobile applications for both Android and iOS using a shared codebase.
- It ensures uniform design and behavior across devices, reduces development time, and simplifies maintenance.

2.4.2. Firebase Services:

- **Authentication:** Provides secure user registration and login functionality, including support for email/password, social logins, and multi-factor authentication.
- **Cloud Firestore:** A scalable, real-time NoSQL database that stores users, attendance records, courses, sessions, disputes, and leave requests.
- **Cloud Functions:** Used to handle backend logic such as sending notifications, session lifecycle handling, and automated timestamp validation.
- **Cloud Storage:** Temporarily stores captured face images or data needed for facial recognition validation.

2.4.3. GPS and Google Maps API:

- Integrated to obtain the student's real-time location and validate whether it falls within the pre-defined geofenced boundary of the classroom.
- The geofence radius (e.g., 50m) is adjustable depending on classroom layout and institutional requirements.

2.4.4. Facial Recognition API (Custom using React Native and Node.js):

- Facial detection and verification is performed on the client using device cameras via React Native's camera capabilities.
- Facial data is temporarily processed on the mobile device, then sent securely to the backend where a Node.js-based facial recognition API compares it against stored biometric templates.
- This dual-processing approach improves performance by leveraging local processing before involving the server.
- Liveness detection and confidence scoring mechanisms help prevent spoofing.

3. System Architecture

3.1. Overview

The system architecture of the Mobile-Based Attendance Management System is designed using a client-server model with a strong emphasis on modularity, scalability, and real-time processing. This architecture supports multiple user roles and provides seamless mobile access, robust geolocation services, and biometric verification capabilities, all underpinned by secure cloud infrastructure.

3.2. Architectural Design

The architecture is logically divided into three layers:

3.2.1. Presentation Layer (Mobile App):

- Developed in React Native to support Android and iOS platforms.
- Responsible for capturing user input, rendering views, and displaying data.
- Interfaces with device hardware (GPS and camera) to collect location and facial recognition data.

3.2.2. Application Logic Layer (Backend Services):

- Implements core business logic through Firebase Cloud Functions and Node.js APIs.

- Processes check-ins, session control, data validation, and notification delivery.
- Ensures integrity and consistency of attendance-related actions.

3.2.3. Data Storage Layer (Firebase Backend):

- Utilizes Firestore for structured, real-time document-based data storage.
- Manages users, courses, sessions, attendance logs, and requests.
- Uses Firebase Cloud Storage for temporary storage of biometric data.

3.3. System Components

3.3.1. Mobile Application:

- User interface for students, lecturers, and administrators.
- Manages session interaction, check-in procedures, and real-time updates.

3.3.2. Authentication Service:

- Firebase Authentication supports login and registration.
- Enforces role-based access and security protocols.

3.3.3. Session Manager:

- Controls course session creation, modification, and closure.
- Prevents duplicate or overlapping sessions.

3.3.4. Location Validator (Geofencing):

- Receives GPS coordinates and validates proximity to the classroom location.
- Returns results to allow or deny attendance check-in.

3.3.5. Facial Recognition Engine:

- Captures facial data via the React Native interface.
- Sends data to a Node.js-based recognition API for processing.
- Processed data is compared with stored profiles and uses liveness detection.

3.3.6. Attendance Logger:

- Stores validated check-ins including timestamps, location match, and confidence score.
- Maintains history and edit logs.

3.3.7. Leave and Dispute Handler:

- Manages student-submitted requests and lecturer responses.

3.3.8. Dashboard and Reporting Engine:

- Displays attendance statistics in different graphs

- Generates downloadable reports.

4. System Modules

This chapter outlines the key modules of the system and how they interact to accomplish the overall functionality. Each module is a self-contained unit responsible for one or more related functions within the system. Technical requirements and key constraints are also discussed.

4.1. User Management Module

- Handles user registration, login, authentication, and role-based access control.
- Integrates with Firebase Authentication to ensure secure credential storage and session management.
- Assigns roles (Student, Lecturer, Admin) and restricts features based on access level.
- Interacts With the Homepage Module (to redirect users, post-login), Notification Module (for verification alerts).
- **Technical Requirement:**
 - Firebase Authentication SDK.
- **Constraint:**
 - Relies on stable internet connection and proper setup of Firebase rules.

4.2. Course and Session Management Module

- Allows lecturers to create, update, and manage course information.
- Enables scheduling of attendance sessions with attributes like course ID, date, time, and location.
- Prevents session duplication and ensures time-bound visibility of sessions for students.
- Interacts With the Attendance Check-in Module, Dashboard Module.
- **Technical Requirement:**
 - Firestore Database for storing course/session metadata.
- **Constraint:**
 - Sessions must be manually initiated by lecturers to ensure attendance is valid.

4.3. Attendance Check-in Module

- Initiates real-time check-in process for students during an active session.
- Captures GPS data from the user's device and validates it against a predefined geofence.
- Launches device camera and uses facial recognition APIs to verify the student's identity.

- Only when both checks are passed, attendance is marked in the database.
- Interacts With the Attendance Logging Module, Notification Module.
- **Technical Requirement:**
 - Access to device GPS and camera, React Native & Node.js Face Recognition API.
- **Constraint:**
 - Device must have location and camera permissions enabled.

4.4. Lecturer-Assisted Check-In Module

- Provides lecturers the ability to bulk scan student faces and verify attendance manually.
- Validates the lecturer's GPS location to ensure the check-in is conducted on-site.
- Displays all recognized students for approval and records attendance in batch.
- Interacts With the Attendance Logging Module, Dashboard Module.
- **Technical Requirement:**
 - Batch face detection support via Node.js facial recognition library.
- **Constraint:**
 - Requires clear visibility and good lighting conditions for group face capture.

4.5. Leave and Dispute Management Module

- Allows students to apply for leave or submit disputes on attendance errors.
- Collects necessary information such as reason, dates, and supporting documents.
- Provides lecturers with an interface to review and act on these requests.
- Updates the attendance record and notifies students of the outcome.
- Interacts With the Notification Module, Attendance Logging Module.
- **Technical Requirement:**
 - Form submission interfaces and Firestore document updates.
- **Constraint:**
 - Requests must be made before the session expires to be considered valid.

4.6. Attendance Logging and History Module

- Records all successful check-ins with timestamp, session ID, location data, and face match score.
- Maintains a historical log of changes made by lecturers, including edit reasons.
- Provides access to personal logs for students and class-wide logs for lecturers.
- Interacts With the Dashboard Module, Reporting Module, Dispute Module.
- **Technical Requirement:**
 - Structured NoSQL design using Firestore collections.
- **Constraint:**
 - Logs are read-only after a certain threshold unless modified by authorized users.

4.7. Dashboard and Reporting Module

- Presents summarized attendance statistics for users based on their role.
- Enables filtering by course, date, and status (present, absent, leave, etc.).
- Allows authorized users to generate downloadable attendance reports in CSV/PDF format.
- Interacts With the Attendance Logging Module, Session Module, User Management.
- **Technical Requirement:**
 - Chart libraries, PDF/CSV export tools.
- **Constraint:**
 - Large datasets may slow down report generation on low-end devices.

4.8. Notification Module

- Sends real-time alerts to students and lecturers about new sessions, request decisions, or changes in schedule.
- Utilizes Firebase Cloud Messaging to push updates to mobile devices.
- Interacts With the all other modules (triggered by session start, login, request handling, etc.).
- **Technical Requirement:**
 - Firebase Cloud Messaging setup and listener integration.
- **Constraint:**
 - Notifications may be missed if the device is offline or app permissions are disabled.

These are the main modules in the system and each of these modules contributes to a modular, maintainable, and scalable system. Their interactions ensure seamless communication across system functionalities.

Firebase's real-time synchronization and cloud functions allow efficient handling of logic and data consistency.

5. UML Diagrams

Unified Modeling Language (UML) diagrams are essential tools used to visually represent the structure and behavior of a system. They help developers, designers, and stakeholders understand how different components of the system interact, what roles each element plays, and how the overall system functions in both static and dynamic views.

In this project, various UML diagrams are used to model specific aspects of the Mobile-Based Attendance Management System. These diagrams provide clarity and alignment throughout the design and development phases.

The following UML diagrams will be included:

1. **Use Case Diagram:** Shows the high-level interactions between system actors (students, lecturers, admin) and system functionalities. It defines what the system does from the user's perspective.
2. **Class Diagram:** Represents the static structure of the system by detailing classes, their attributes, methods, and relationships. It helps in understanding object-oriented organization and dependencies.
3. **Context Diagram:** A high-level view of the system that shows the system as a single entity and its interactions with external actors or systems.
4. **Data Flow Diagram (DFD):** Illustrates the flow of data within the system, depicting sources, processes, data stores, and destinations. It is useful for understanding how data is processed and transferred.
5. **Activity Diagram:** Visualizes the flow of activities and operations within the system. It is especially useful for describing processes like user check-in, leave submission, and course session creation.
6. **Sequence Diagram:** Displays the sequence of messages exchanged between components or users over time. It highlights the chronological order of interactions for specific processes.
7. **Deployment Diagram:** Illustrates the system's physical deployment environment, including hardware nodes, software components, and their communication paths. It supports decisions related to system infrastructure and hosting.

5.1. Use Case Diagram

5.1.1. Diagram

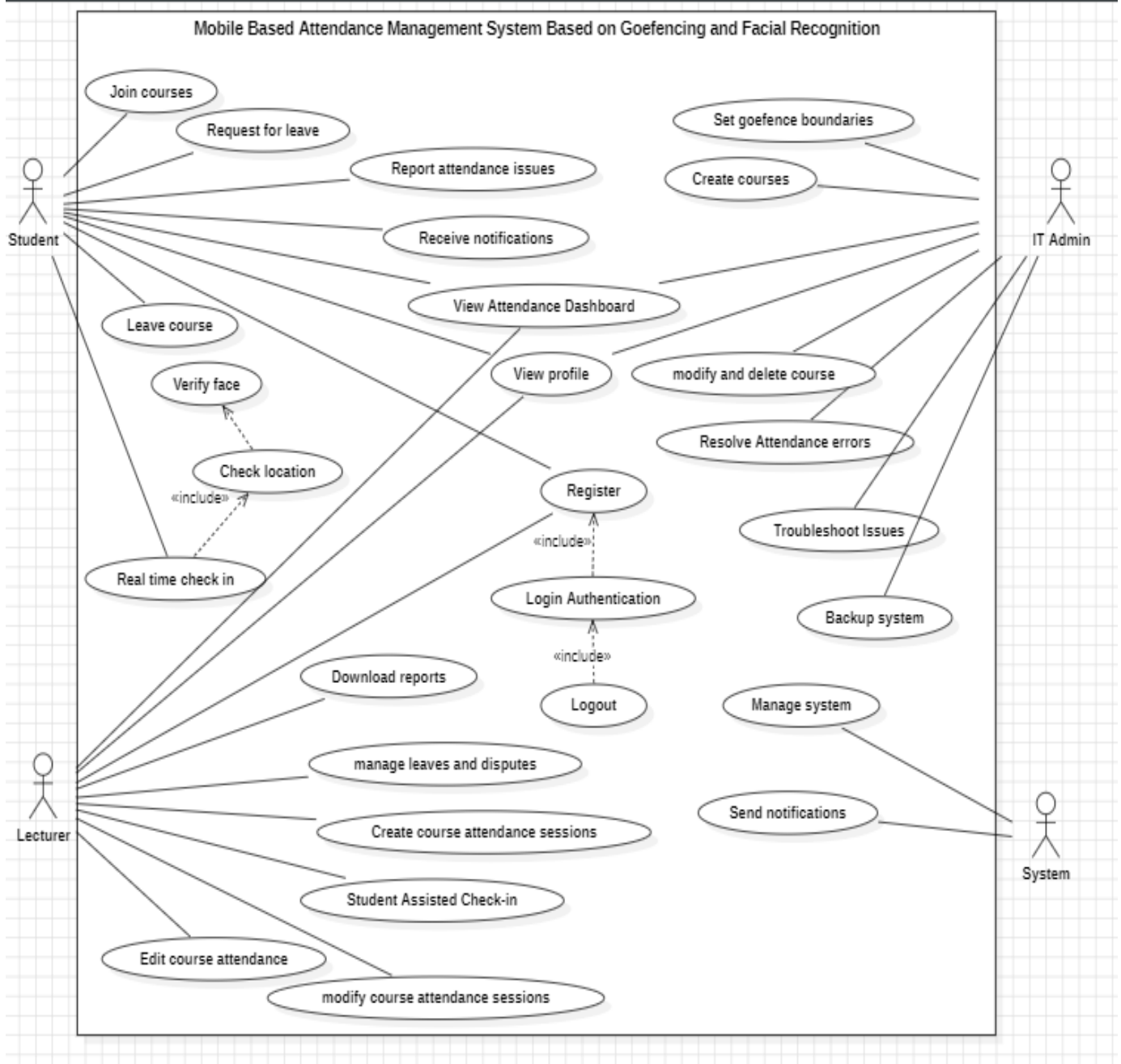


Figure 1: Use case diagram for the System

5.1.2.Use Case description

5.1.2.1. Register / Login

- **Actors:** Student, Lecturer, Admin
- **Preconditions:** The user has installed the mobile application. If registering, the user must provide unique credentials. If logging in, they must already have a registered account.
- **Triggers:** The user launches the application and selects either "Register" or "Login" on the welcome screen.
- **Main Flow:**
 - i. The user selects "Register" or "Login".
 - ii. For registration:
 - The user provides their full name, email, password, role (student, lecturer, or admin), and optionally a phone number.
 - The system checks if the email is already registered.
 - If unique, the system stores the user data using Firebase Authentication and creates a user profile document in Firestore.
 - iii. For login:
 - The user inputs their registered email and password.
 - Firebase Authentication verifies the credentials.
 - iv. Upon successful login, the system generates a secure token and redirects the user to their role-specific dashboard.
- **Alternative Flows:**
 - i. If login fails due to incorrect credentials, the user receives an error message prompting a retry.
 - ii. If the user attempts to register with an already used email, they are advised to use the login option instead.
- **Postconditions:**

A registered user is securely logged in and granted access to the system features based on their role.

- **Priority:** High

5.1.2.2. Join Courses

- **Actor:** Student
- **Preconditions:** The student is logged into the system and has a valid profile.
- **Trigger:** The student taps the “+” button and input the code for a course given to them by their lecturer and joins the course.
- **Main Flow:**
 - i. The student taps on the "+" button feature on the homepage.
 - ii. A form to input the course code is open.

- iii. The student inputs the correct course code and joins the course.
- iv. The system verifies if the student is already enrolled in the course.
- v. If not enrolled, the system updates the student's profile and the course roster to reflect the new enrollment.
- vi. The student receives a confirmation notification of successful enrollment.

➤ **Alternative Flows:**

- i. If the course code stored in the database is not valid, the system displays an error message.
- ii. If the student is already enrolled, the system disables the join option.

➤ **Postconditions:**

The student is officially enrolled in the course, and the course is now visible on their Homepage.

➤ **Priority:** High

5.1.2.3. Real-Time Check-In

➤ **Actor:** Student

➤ **Preconditions:** The student must be logged in, enrolled in the course, and an active attendance session must be open. The student's mobile device must have GPS and camera permissions enabled.

➤ **Trigger:** The student navigates to an enrolled course and selects the "Check-In" button during an active session.

➤ **Main Flow:**

- i. The system checks if an attendance session is active for the selected course.
- ii. The app requests and verifies the user's current GPS location.
- iii. The location is compared against the predefined geofence coordinates.
- iv. If the student is within the geofence, the system activates the device's front-facing camera.
- v. The camera captures a live facial image of the student.
- vi. The image is processed by the in-device facial recognition api and the processed data is sent to the database and compared with the saved facial template
- vii. If the face matches the student's stored template, the system logs the attendance with a timestamp, GPS coordinates, and face match confidence score.

viii. A success message is displayed to the student.

➤ **Alternative Flows:**

- i. If GPS is turned off, the system prompts the user to enable it.
- ii. If the student is outside the geofence, the system denies check-in and shows a location error.
- iii. If the facial match fails, the system prompts the user to retry or meet their lecturer for Student assisted check-in.

➤ **Postconditions:**

The student's attendance is successfully recorded in Firestore and the attendance record includes time, location, and biometric confirmation.

➤ **Priority:** High

5.1.2.4. Request for Leave

➤ **Actor:** Student

➤ **Preconditions:** The student must be logged in and enrolled in at least one course. The current date must be within the course duration.

➤ **Trigger:** The student navigates to the course page and selects "Request Leave" for a specific session or date.

➤ **Main Flow:**

- i. The student selects the course they will miss.
- ii. They enter the leave reason, start and end date (if applicable), and compulsory attachment or comment.
- iii. The system records the request in the "leave_requests" collection in Firestore.
- iv. A notification is sent to the respective lecturer assigned to the course.
- v. The lecturer logs in and opens the pending requests tab.
- vi. The lecturer views the request details and chooses to approve, deny, or request more information.
- vii. The system updates the request status and notifies the student.

➤ **Alternative Flows:**

- i. If the lecturer does not act on the request, it remains in a pending state and is shown as awaiting action.

- ii. If the student enters invalid dates or submits incomplete information, the system rejects the request and prompts correction.

➤ **Postconditions:**

- i. The request is saved and tracked with a status: Approved, Denied, or Pending.
- ii. The student is notified of the lecturer's decision and any comments provided.

➤ **Priority:** Medium

5.1.2.5. View Attendance Dashboard

➤ **Actors:** Student, Lecturer

➤ **Preconditions:** User must be logged in and enrolled in at least one course.

➤ **Trigger:** User navigates to Dashboard and selects a course then selects "Attendance Analytics" from the menu.

➤ **Main Flow:**

- i. The app fetches attendance data from Firestore based on the user's role.
- ii. For students:
 - The dashboard displays personal attendance statistics, such as present %, absent %, and last check-in.
 - Data can be filtered by course or date.
- iii. For lecturers:
 - The dashboard shows attendance summaries for that course
 - The lecturer can view session-based records.
- iv. The system displays the results using interactive elements like bar charts, pie graphs, and sortable tables.

➤ **Alternative Flows:**

- i. If no data is found for selected filters, the system displays a placeholder or message such as "No attendance records found".

➤ **Postconditions:**

Users gain real-time visual insights into attendance patterns and individual performance.

➤ **Priority:** Medium

5.1.2.6. Create Course Attendance Session

➤ **Actor:** Lecturer

➤ **Preconditions:** Lecturer is authenticated and owns at least one course.

➤ **Trigger:** Lecturer taps "Create Attendance Session" button.

➤ **Main Flow:**

- i. Lecturer selects the course from their assigned list.
- ii. Lecturer specifies session date, time, and location.
- iii. The system checks that no session is already active for that course.
- iv. The system creates a session object in Firestore and marks the status as "OPEN".
- v. Notifications are pushed to all enrolled students, informing them that a check-in session is active.

➤ **Alternative Flows:**

- i. If a session is already active for the same course and date, the system prevents duplication.
- ii. If an invalid course detail is inputted, the system displays an error.

➤ **Postconditions:**

The session is created, listed on both student and lecturer dashboards, and ready for check-ins.

➤ **Priority:** High

5.1.2.7. Student Assisted Check-In

➤ **Actor:** Lecturer

➤ **Preconditions:** Lecturer must be logged in, have an open session, and be physically present in the geofenced location.

➤ **Trigger:** Lecturer chooses a course and taps "Student Assisted Check-In".

➤ **Main Flow:**

- i. The system validates that the session is currently open.
- ii. GPS access is verified and the lecturer's location is checked against the course's geofence.
- iii. Once validated, the camera interface is activated.
- iv. The system scans multiple student faces at once.
- v. Facial recognition identifies matching students and logs them as "Present".

- vi. Lecturer is shown a list of matched students and can manually approve or adjust statuses.
- vii. Final attendance records are saved in Firestore.

➤ **Alternative Flows:**

- i. If GPS is disabled or the lecturer is outside the valid location, the check-in is blocked.
- ii. If face detection fails for some students, they are flagged for manual attendance entry.

➤ **Postconditions:**

Multiple students are checked in by the lecturer using automated facial verification.

➤ **Priority:** High

5.1.2.8. Edit Course Attendance

➤ **Actor:** Lecturer

➤ **Preconditions:** Lecturer must be assigned to the course and the session must not be locked or archived.

➤ **Trigger:** Lecturer navigates to attendance history and selects a session.

➤ **Main Flow:**

- i. Lecturer opens session record.
- ii. Taps on the edit button on the screen to edit student session attendance
- iii. Selects a student and modifies their status (e.g., changes Absent to Present).
- iv. The update is saved to Firestore.
- v. Notifications are sent to the student about their edited attendance.
- vi. An audit trail entry is created showing who made the change and when.

➤ **Alternative Flows:**

- i. If the session is already closed for editing (e.g., older than 7 days), the system denies the modification.
- ii. Unauthorized users attempting edits are denied access.

➤ **Postconditions:**

The attendance record is updated and history is traceable for accountability.

➤ **Priority:** Medium

5.1.2.9. Manage Leaves and Disputes

- **Actor:** Lecturer
- **Preconditions:** One or more students have submitted leave or attendance dispute requests.
- **Trigger:** Lecturer opens "Leave & Disputes" panel
- **Main Flow:**
 - i. System loads all pending requests assigned to the lecturer.
 - ii. Lecturer reviews each request with details (student name, course, reason, timestamp).
 - iii. Lecturer may approve, reject, or send feedback requesting clarification.
 - iv. System updates the request record in Firestore with a status and comment.
 - v. Notification is sent to the student with the outcome.
- **Alternative Flows:**
 - i. If the status of a leave is marked as "Pending", a student can cancel the request
 - ii. Notifications should be sent for changes
- **Postconditions:**

All processed requests are saved and displayed in the request history for both parties.
- **Priority:** Medium

5.1.2.10. Download Reports

- **Actors:** Lecturer, Admin
- **Preconditions:** User must be authenticated and have access to at least one course with recorded attendance data.
- **Trigger:** User selects "Reports" and chooses a format to export.
- **Main Flow:**
 - i. User selects desired course and date range.
 - ii. The system queries attendance logs and compiles data.
 - iii. The user chooses between export formats: CSV or PDF.
 - iv. The report is generated and a download prompt appears.
 - v. File is saved locally or shared via supported apps.
- **Alternative Flows:**
 - i. If no data matches the selected filters, a "No data available" message is shown.

ii. If the export fails (due to timeout or server issue), user is prompted to retry.

➤ **Postconditions:**

The user successfully receives a detailed attendance report for analysis or record-keeping.

➤ **Priority:** Medium

5.2. Class Diagram

The Class Diagram outlines the static structure of the system, defining the classes, their attributes, methods, and relationships.

5.2.1. Diagram

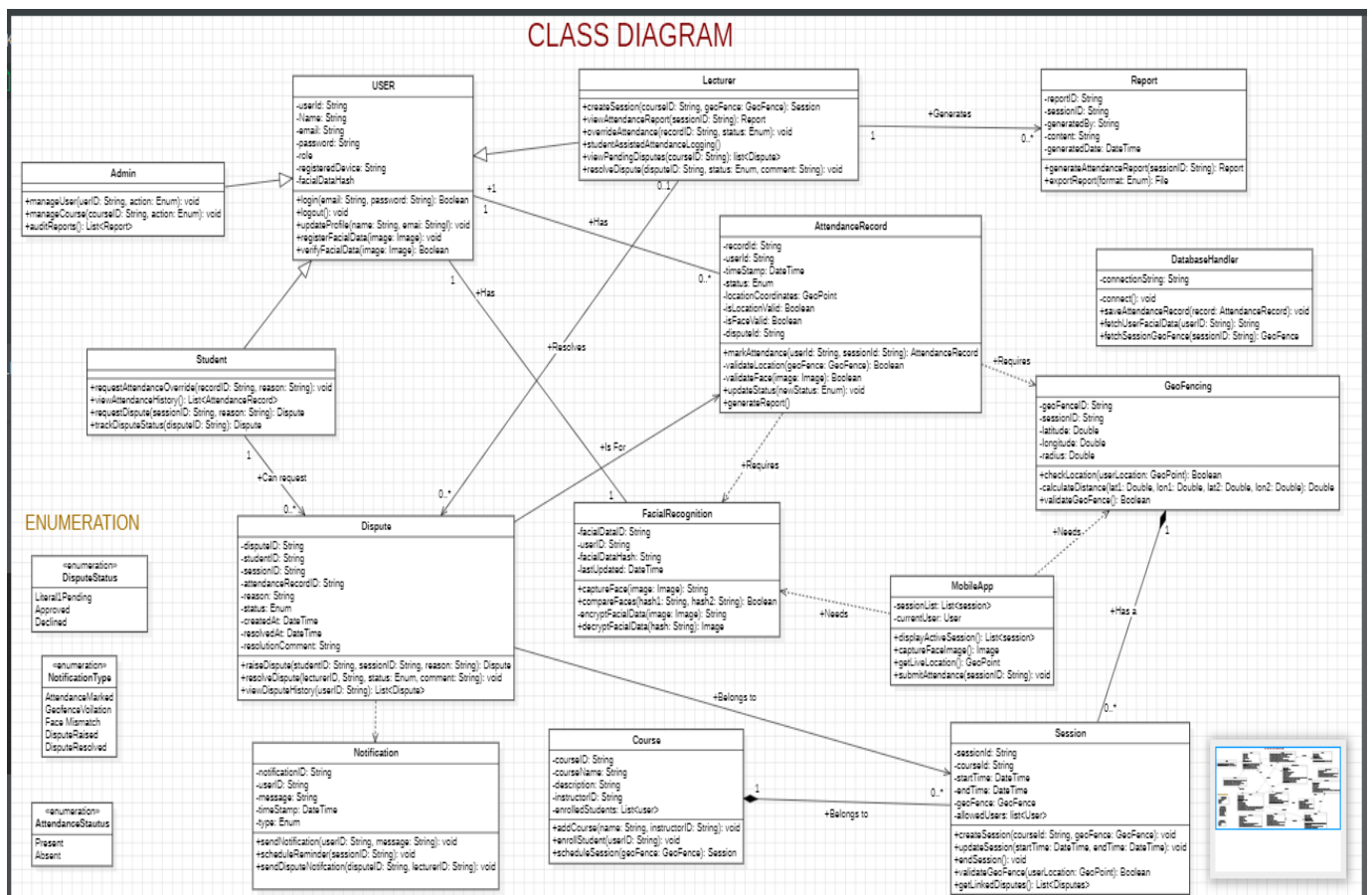


Figure2: Class Diagram for the Mobile Attendance System

5.2.2. Brief Explanation of Key Classes

➤ **User (Abstract):** This is a foundational abstract class representing any individual interacting with the system. It holds common attributes like `userId`, `username`, `passwordHash`, `firstName`, `lastName`, `email`, and `role`. It also defines common methods like `login()`, `logout()`, and `updateProfile()`.

- **Student:** Inherits from User. Represents a student. It includes student-specific attributes like `studentId`, `enrollmentNumber`, and a link to their `FacialData`. Key methods include `registerFacialData()`, `markAttendance()`, and `raiseDispute()`.
 - **Lecturer:** Inherits from User. Represents a faculty member. Attributes include `facultyId` and `department`. Key methods involve managing courses and sessions (`createCourse()`, `scheduleSession()`), viewing attendance, and `resolveDispute()`.
 - **Admin:** Inherits from User. Represents a system administrator with methods for user management (`createUser()`, `deleteUser()`), system settings, and managing geofences.
- **Course:** Defines an academic course with attributes like `courseCode`, `courseName`, and a reference to the Lecturer teaching it. It manages lists of enrolled students and course sessions.
 - **CourseSession:** Represents a specific lecture or lab session for a Course, including `startTime`, `endTime`, `date`, `locationDescription`, and a link to the `GeofenceLocation`. It also tracks its status (e.g., OPEN, CLOSED).
 - **AttendanceRecord:** Links a Student to a CourseSession, storing the timestamp of attendance, the status (Present, Absent, etc.), and the `verificationMethod`.
 - **FacialData:** Stores the biometric embeddings (facial features) for a Student, used for facial recognition.
 - **GeofenceLocation:** Defines a geographical boundary with `centerLatitude`, `centerLongitude`, and `radiusInMeters`, used to validate attendance location.
 - **Dispute:** Allows Students to raise issues regarding their attendance for a CourseSession. It contains `reason`, `details`, `status` (Pending, Approved, Declined), and is resolved by a Lecturer.
 - **Notification:** An optional but important class for sending alerts to users about various events like dispute status updates or session reminders.
 - **Enumerations (`UserRole`, `AttendanceStatus`, `DisputeStatus`, `SessionStatus`):** These define sets of predefined constant values for attributes within the main classes, ensuring consistency and readability.

Relationships

The diagram illustrates;

- Inheritance (e.g., *Student is a User*),
- aggregation/composition (e.g., *Course has CourseSessions*), and
- associations (e.g., *Student raises Dispute*, *Lecturer resolves Dispute*).

5.3. Context Diagram

The context diagram provides a high-level view of the entire system as a single process and defines the boundaries between the system and its users. It shows how the three main external entities (Students, Lecturers, and Admins), interact with the Attendance Management System, and what types of information flow between them. This helps provide a simple yet comprehensive understanding of how the system fits within its environment.

5.3.1. Diagram

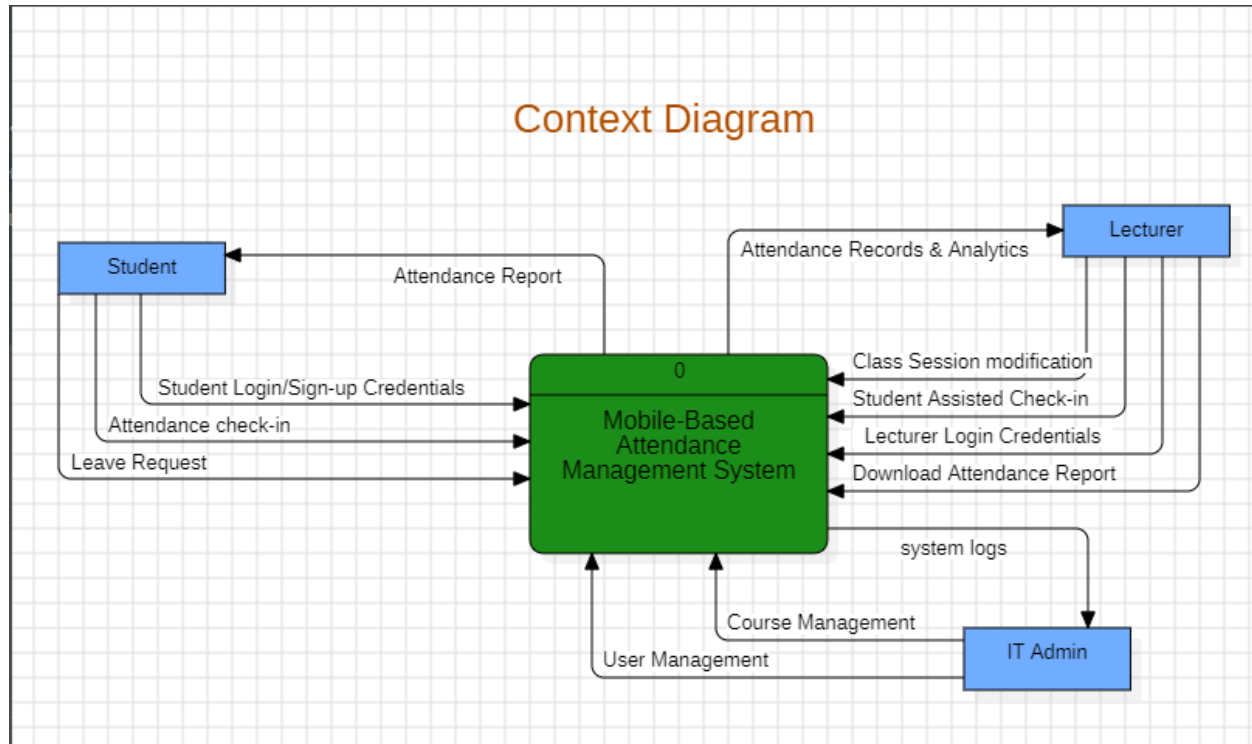


Figure 3: Context Diagram

5.3.2. System Description:

The **Mobile-Based Attendance Management System** is the central system responsible for managing user authentication, attendance session control, student check-in, leave requests, dispute management, and reporting. It relies on data provided by users and returns outputs that support class attendance monitoring and recordkeeping.

External Entities:

- **Students** use the system to register and log in, join courses, check in to class sessions using geolocation and facial recognition, submit leave requests or disputes, and view their attendance records. The system receives input from students such as login credentials,

check-in attempts (with location and facial data), and leave request forms. It responds with attendance results, session availability, and notifications.

- **Lecturers** use the system to create and manage attendance sessions, perform student-assisted check-ins, review and edit attendance records, and respond to student disputes or leave requests. The system accepts inputs such as session creation commands, attendance edits, and dispute approvals. It provides lecturers with real-time student attendance updates and access to historical reports.
- **Admins** (optional, depending on scope) manage the user base, assign roles, and oversee reporting and data integrity. Admins typically perform backend tasks like updating user roles, monitoring system logs, and exporting attendance reports across courses or institutions.

5.3.3. Major Interactions:

- **Students** send data like login credentials, GPS coordinates, face images, and leave request forms to the system. The system returns authentication results, check-in confirmations, attendance history, and notifications.
- **Lecturers** interact by initiating attendance sessions, viewing attendance records, resolving disputes, and downloading reports. The system responds with confirmation of session status, attendance statistics, and alert messages.
- **Admins** perform administrative tasks such as role assignment, user oversight, and system monitoring. The system provides user records, logs, and generated reports in response to these interactions.

5.3.4. System Boundary:

The central block of the context diagram is the **Mobile-Based Attendance Management System**. This block is the only component within the system boundary. All three user types (Students, Lecturers, and Admins) are external to the system. Their interactions occur through defined interfaces (e.g., Homepage, dashboards) and are modeled as data flowing into and out of the system.

Notes:

- This context diagram does not reflect internal processes or components, those are captured in the Data Flow and System Architecture diagrams.
- Only three core entities are considered external actors in this design: Students, Lecturers, and Admins.
- All external interactions are expected to be secure, authenticated, and role-based.

5.4. Data Flow Diagram

This Data Flow Diagram (Level 1) illustrates the core functional processes of the attendance management system, detailing how data flows between students, lecturers, external services (like GPS and Facial Recognition), and the system's internal components. It focuses on the processes involved in registering users, managing course sessions, checking attendance, and downloading reports.

5.4.1. DFD Level 1 Diagram

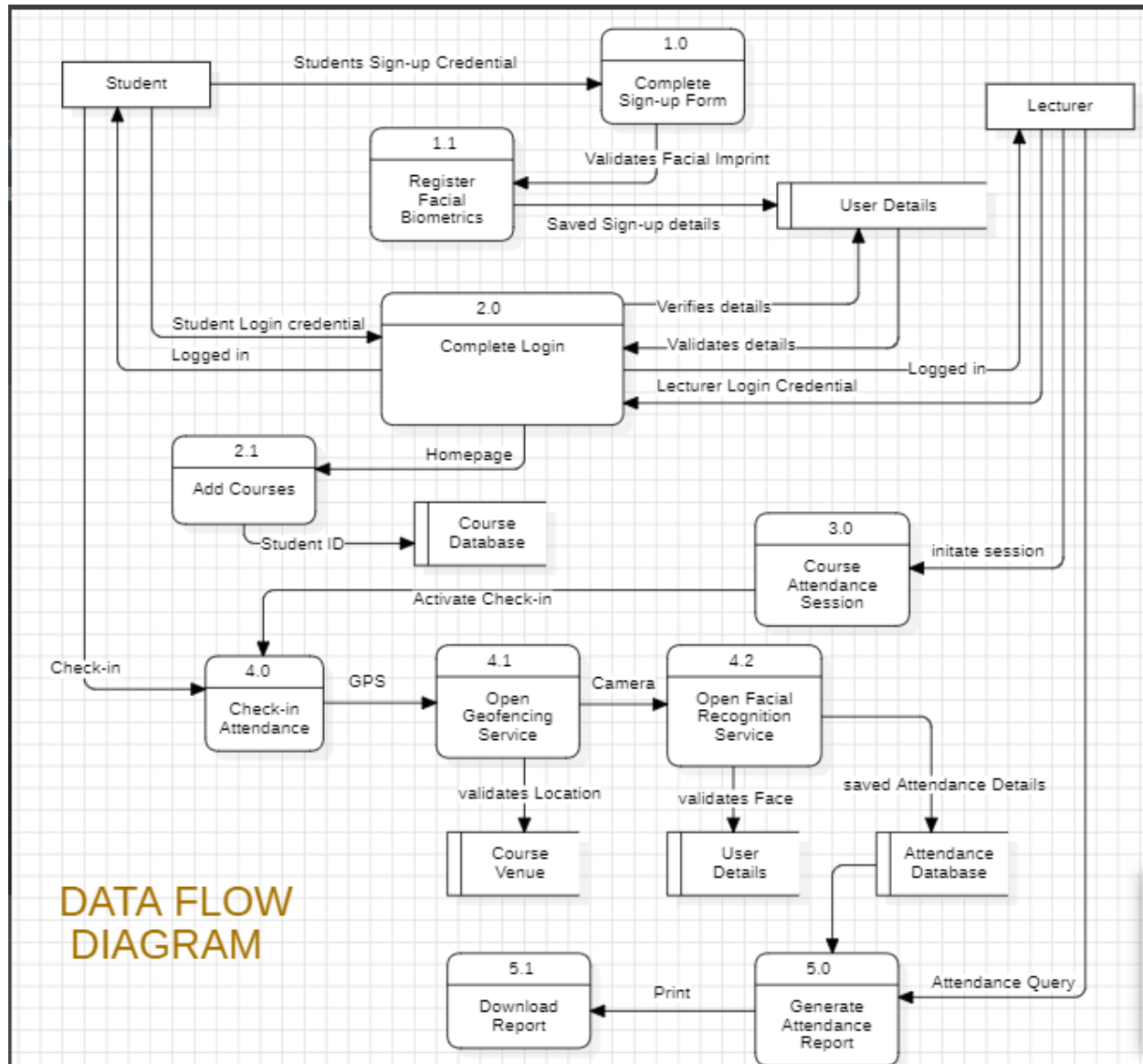


Figure 4: Data Flow Level 1 Diagram

5.4.2. Processes and Descriptions

Process ID	Process Name	Description
1.0	Complete Sign-up Form	Handles registration. Saves user credentials and grants access based on roles (Student, Lecturer, Admin).
1.1	Register Facial Biometrics	Enables the user to register their facial data to the database as a template to be used for Check-in.
2.0	Complete Login	Validates User details from the database and gives them access to their respective role-based homepage
2.1	Add Course	Student Enrolls into the course of their choice by imputing a specific course code for a particular course and their names saved to the database
3.0	Course Attendance Session	Lecturer creates a specific course attendance session for a particular course which enables the check-in module for the students and Lecturers to be OPENED.
4.0	Check-in Attendance	Manages the student check-in process. Verifies GPS location within geofence and uses the face recognition module to confirm identity. Only if both validations pass is attendance logged.
4.1	Open Geofencing Service	Verifies GPS location within geofence
4.2	Open Facial Recognition Service	Stores verified attendance entries into the Firebase database with metadata (timestamp, session ID, location, face match score).
5.0	Generate Attendance Report	Retrieves attendance data for. Supports filtering, summary stats, and report generation.
5.1	Download Report	Enables Lecturers and Admins to Download attendance and System Log reports for students and System activities Respectively

5.4.3. External Entities

Entity	Description
Student	Main user who checks in to sessions, submits leave/dispute requests, and views attendance history.
Lecturer	Manages course sessions, reviews disputes, and monitors attendance reports.
GPS Service	Provides geolocation data to verify whether a student is within the allowed attendance area (geofence).
Face Recognition API	Processes facial images for identity verification during student check-in.

5.4.4. Data Stores

Data Store	Contents
User Details	Contains user profiles, roles, and authentication records.
Course & Session Store	Stores course metadata, session schedules, geofencing data, and session status (open/closed).
Attendance Database	Stores each attendance record with location, StudentID, facial verification result, and timestamp.

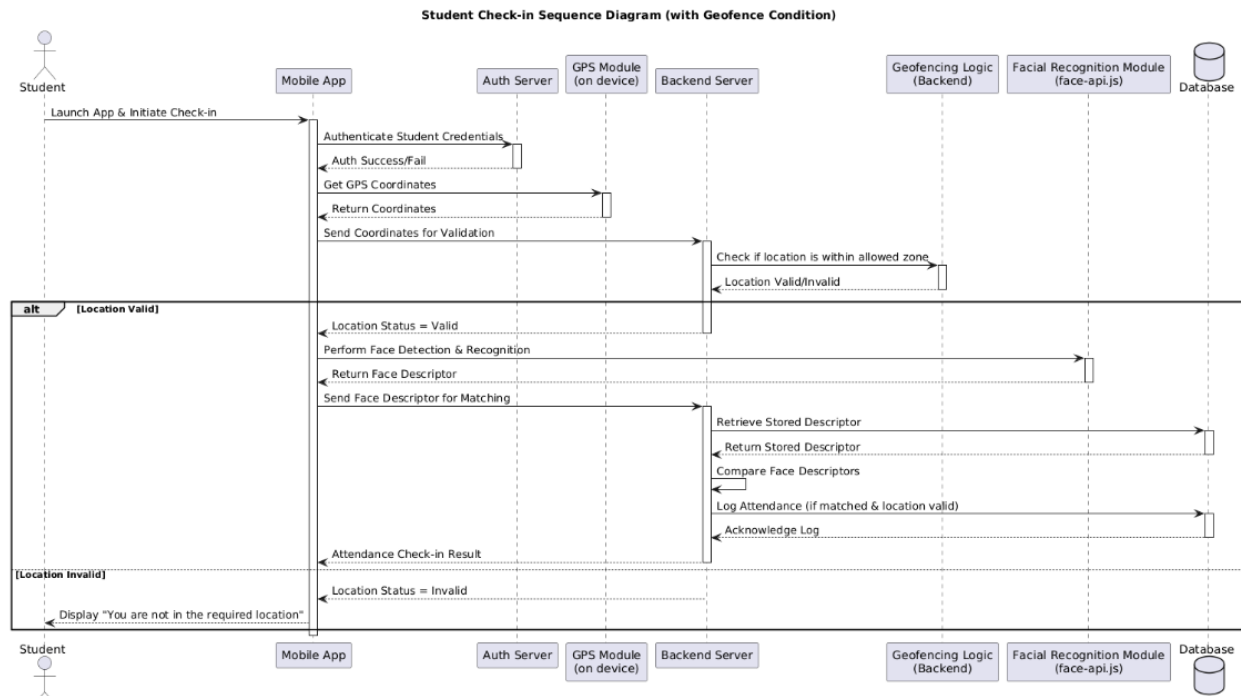
5.5. Sequence Diagrams

5.5.1. Student Attendance Check-in

5.5.1.1. Purpose:

To illustrate how a student uses the mobile app to check in for attendance, with secure authentication, geolocation validation, and facial recognition.

5.5.1.2. Diagram



5.5.1.3. Participants:

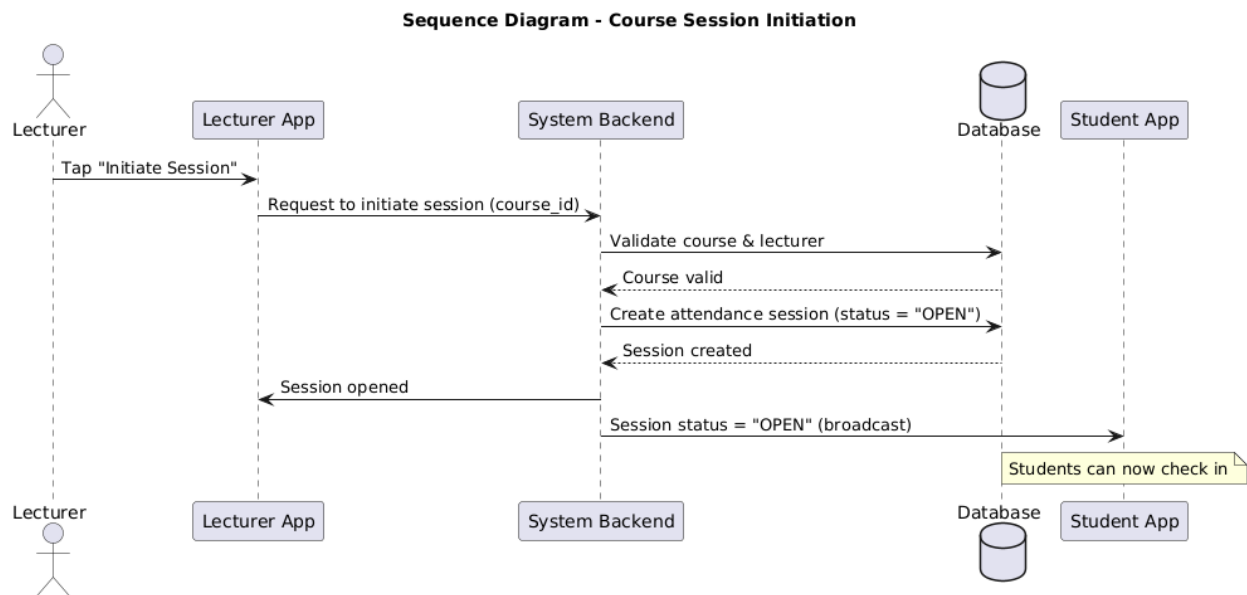
Actor/Component	Description
Student	User initiating the check-in via the mobile app
Mobile App	UI handling user actions and coordinating requests
Auth Server	Validates student login credentials
GPS Module (on device)	Captures the device's location
Backend Server	Core logic that verifies geofence and facial data
Geofencing Logic	Validates if location is within allowed boundary
Facial Recognition Module (face-api.js)	Detects and verifies the student's face
Database	Stores student profiles and attendance logs

5.5.1.4. Sequence Description

- i. Student launches the app and initiates check-in.
- ii. Mobile App authenticates the student via the Auth Server.
- iii. GPS module retrieves the current coordinates.
- iv. Mobile App sends coordinates to the Backend Server.
- v. Backend invokes Geofencing Logic to verify location.
- vi. If valid, Facial Recognition is triggered via face-api.js (on the client).
- vii. Mobile App sends the facial descriptor to the Backend.
- viii. Backend fetches the stored descriptor from the Database.
- ix. Backend compares descriptors and logs attendance if they match.
- x. Result is returned to the Student via the Mobile App.

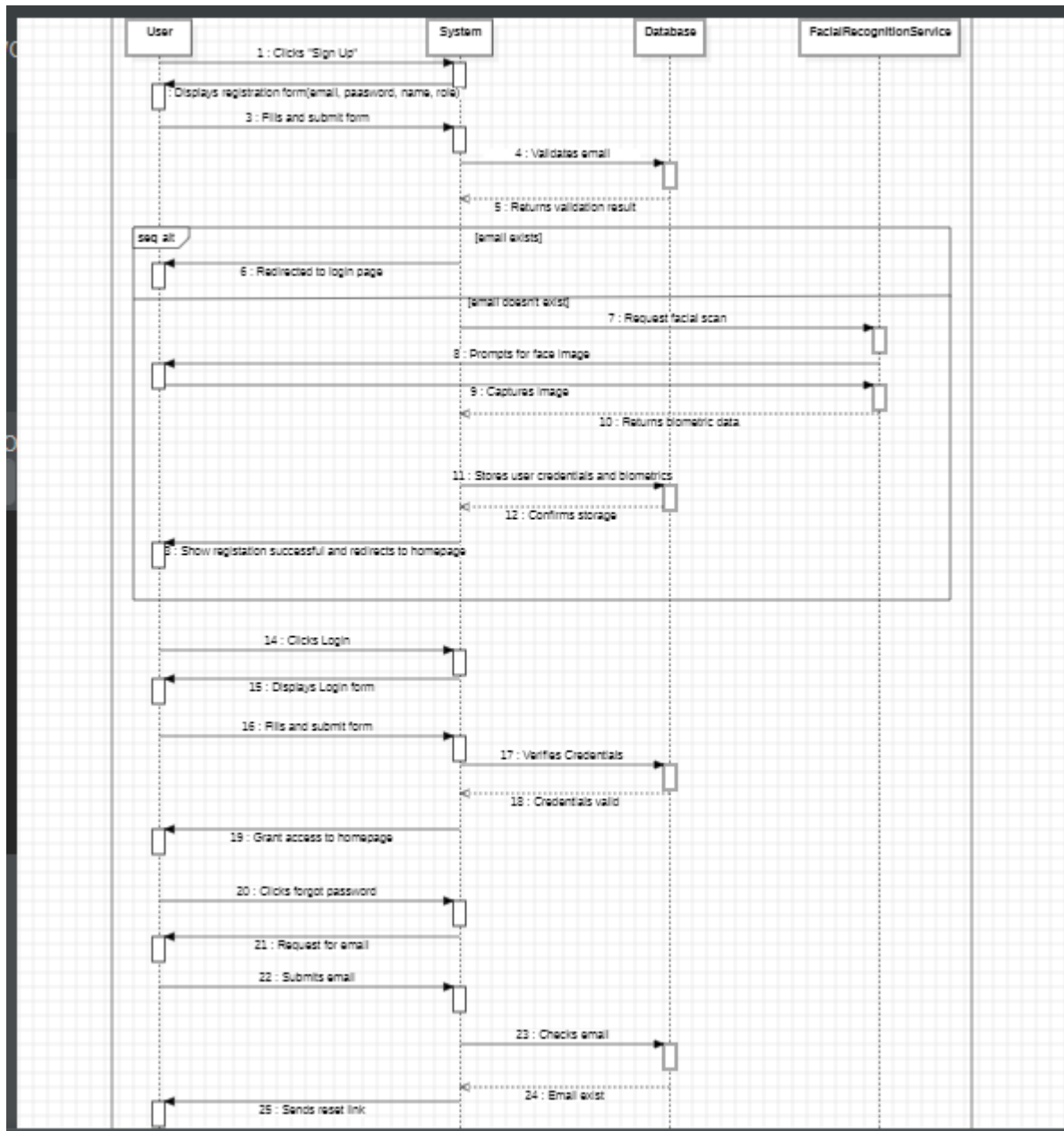
5.5.2. Course Session Initiation

5.5.2.1. Diagram



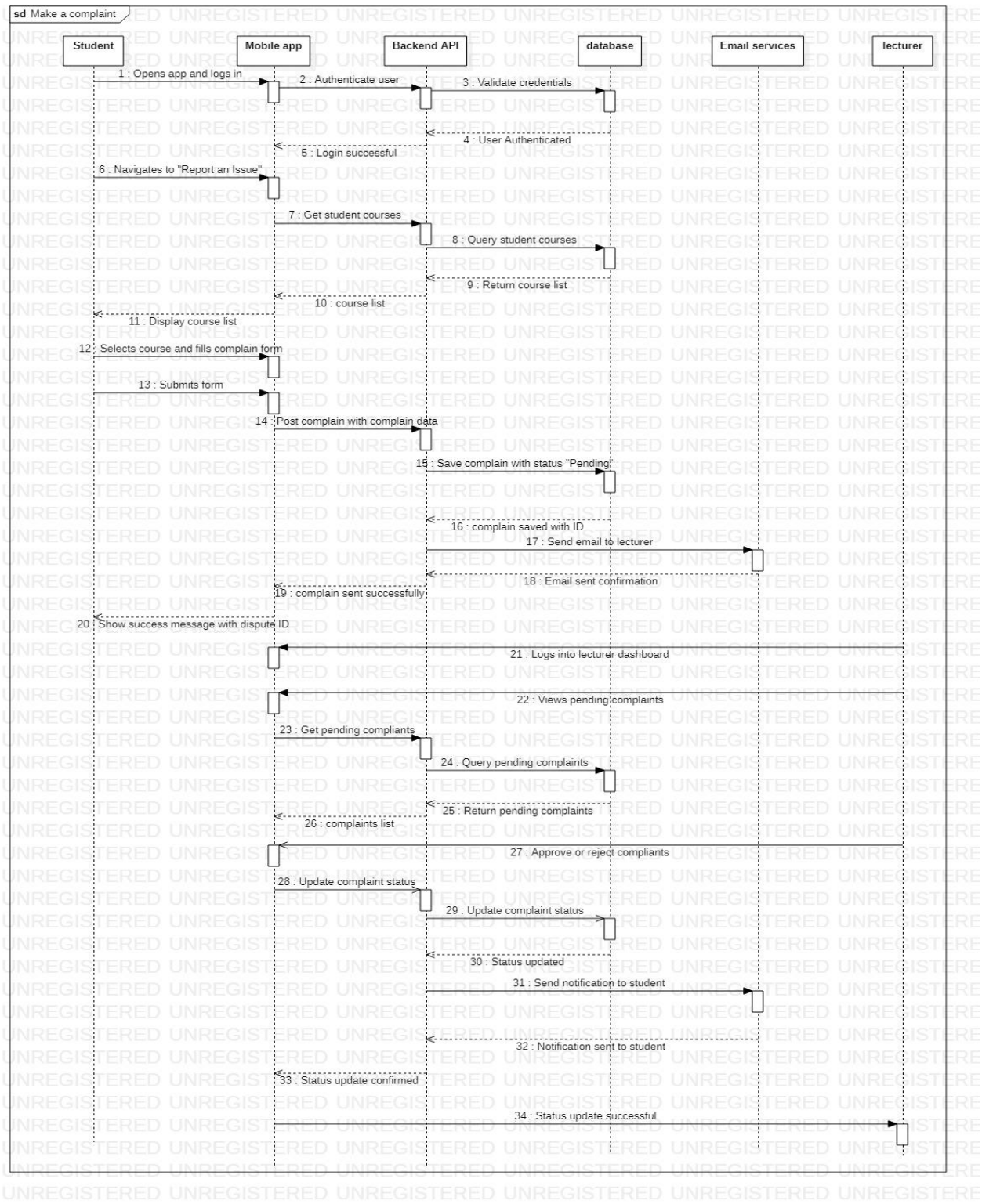
5.5.3. Sign-up/Login

5.5.3.1. Diagram



5.5.4. "Make a Complaint"

5.5.4.1. Diagram



5.5.4.2. Process Explanation

➤ Submission

- Student submits complaint through mobile app
- Complaint is saved to Firebase under /complaints/[courseID]

➤ Notifications

- Automatic email sent to lecturer
- Instant push alert to admin devices

➤ Resolution

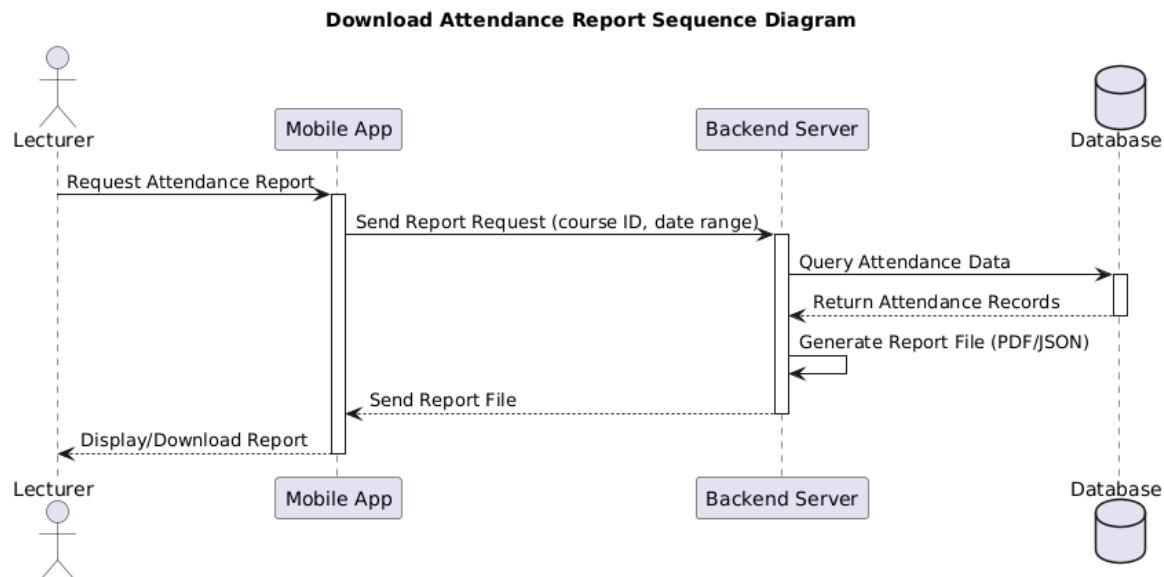
- Lecturer reviews and updates status
- Student sees real-time update (<0.5 sec delay)

5.5.5. Download Attendance Report

5.5.5.1. Purpose

To illustrate how a lecturer downloads attendance reports via the mobile app, showing internal data access and report generation.

5.5.5.2. Diagram



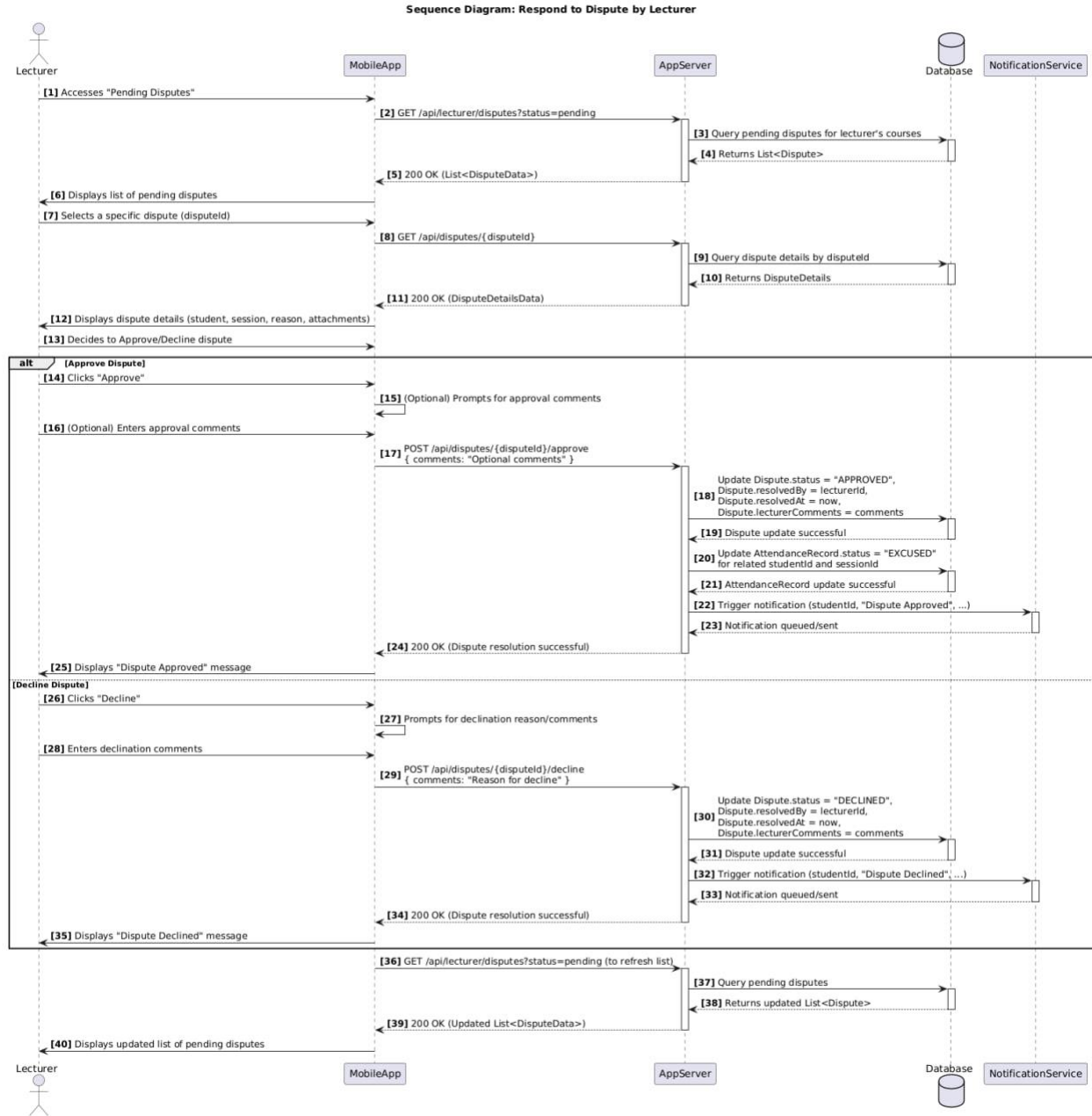
5.5.5.3. Participants

Actor/Component	Description
Lecturer	User requesting the report
Mobile App	Interface for submitting and receiving the report
Backend Server	Processes the request and compiles the report
Database	Stores attendance data to be queried

5.5.5.4. Sequence Description

- i. Lecturer opens the Mobile App and requests a report.
- ii. Mobile App sends the request to the Backend Server.
- iii. Backend queries the Database for relevant attendance data.
- iv. Data is returned and compiled into a formatted report (e.g., PDF).
- v. Backend sends the file to the Mobile App.
- vi. Mobile App displays or prompts the Lecturer to download the report.

5.5.6. Respond to Dispute by Lecturer



Respond to Dispute by Lecturer Sequence Diagram

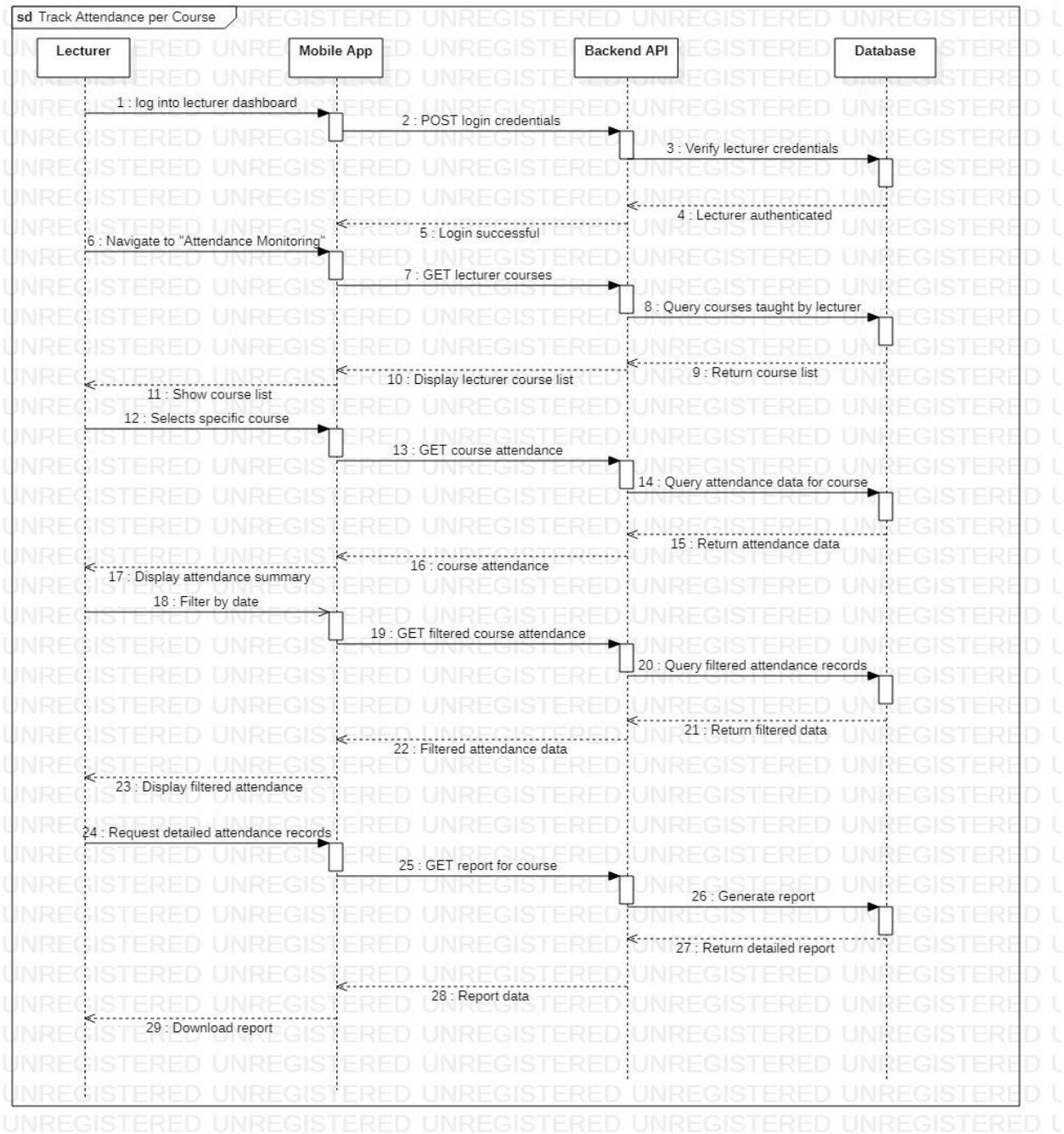
5.5.6.1. Brief Explanation of Flow

This diagram shows the detailed message flow when a lecturer resolves a dispute.

- i. The **Lecturer** logs into the **Mobile App**. The App communicates with the **System Backend** for authentication, which in turn interacts with the **Database (DB)**.
- ii. The Lecturer requests to view pending disputes. The App sends a request to the Backend, which queries the DB and returns the list.
- iii. The Lecturer selects a specific dispute. The App requests details from the Backend, which retrieves them from the DB.
- iv. The Lecturer decides to approve or decline the dispute via the App. The App sends a resolution request (e.g., PUT request) to the Backend.
- v. The **Backend** validates the request.
- vi. **If Approve:** The Backend instructs the DB to update the dispute's status and the corresponding student's attendance record. It then triggers a notification to the student.
- vii. **If Decline:** The Backend instructs the DB to update the dispute's status and triggers a notification.
- viii. The Backend sends a confirmation back to the App, which updates the UI for the Lecturer.
- ix. The App may refresh the list of pending disputes by making another GET request to the Backend.

5.5.7. "Track Attendance per Course"

5.5.7.1. Diagram



5.5.7.2. Process Explanation

- **Lecturer Request:** Opens attendance dashboard.
- **Firestore Query:** Mobile app fetches data from Firestore (filtered by course/date).
- **Real-Time Updates:** Firebase listeners auto-refresh the UI when data changes.

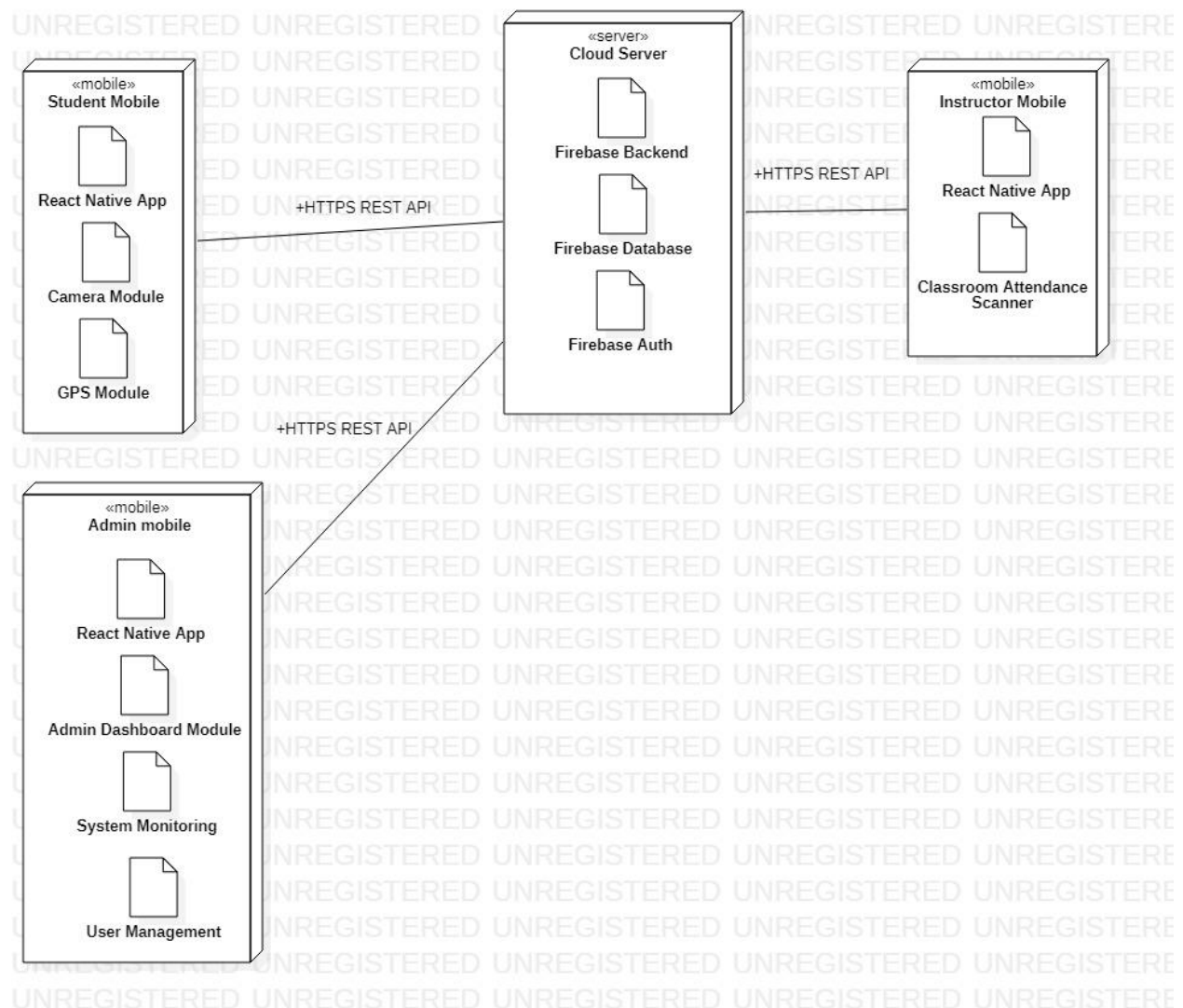
5.6. Deployment Diagram

The system consists of:

- **Mobile App (Android/iOS):** Built with React Native for cross-platform compatibility.
- **Firebase Backend:** Handles authentication, data storage, and real-time updates.
- **Facial Recognition Module:** On-device face verification (using face api .js).
- **Geofencing Module:** Validates student location via GPS.

5.6.1. Diagram Representation

Below is the Deployment Diagram illustrating the system's physical and software components:



5.6.2. Key Components

Component	Role
Mobile Device	Hosts the app; runs facial recognition and geofencing.
Firebase Backend	Manages authentication, real-time data (Firestore), and cloud functions.
Firestore Database	Stores attendance records, user profiles, and geofence coordinates.
Firebase Authentication	Handles secure login (email/password, biometrics).
Facial Recognition	On-device AI model for identity verification.
Geofencing Module	Validates student location before marking attendance.

5.6.3. Communication Protocols

- HTTPS: Secure API calls between mobile app and Firebase.
- Web Sockets: Real-time sync for attendance monitoring (Firestore listeners).

5.7. Activity Diagrams

Activity diagrams model the dynamic aspects of the system, showing the flow of control from one activity to another. They are excellent for visualizing workflows and business processes.

5.7.1. Student Attendance Check-in (Activity Diagram)

5.7.1.1. Purpose

To model the **step-by-step flow of actions and decisions** that a student goes through when performing an attendance check-in using GPS and facial recognition.

5.7.1.2. Diagram



5.7.1.3. Actors Involved

- **Student** (User interacting with the app)
- **Mobile App** (Client-side logic and interface)
- **Auth Server** (Authenticates login credentials)
- **Backend Server** (Validates location and face)
- **face-api.js** (Client-side facial recognition)
- **Database** (Stores attendance records)

5.7.1.4. Flow Description

- i. **Start** – The process begins when the student launches the mobile application.
- ii. **Authentication** – The student logs in using credentials validated by the Auth Server.
- iii. **Navigation** – The student proceeds to the *Attendance Login* page within the app.
- iv. **GPS Location Capture** – The mobile app uses the device's GPS module to obtain the student's current location.
- v. **Geofence Validation** – The location coordinates are sent to the backend, where the backend determines if the student is within the allowed geofence for that course.
- vi. **Decision – Inside Geofence?**
 - **Yes:**
 - The camera is activated.
 - face-api.js performs liveness detection and extracts a face descriptor.
 - The descriptor is sent to the backend.
 - The backend fetches the student's reference descriptor from the database.
 - The two descriptors are compared.
 - If they match: attendance is logged in the database.
 - If they don't match: an error message is shown ("Face not recognized").
 - **No:**
 - An error message is displayed to the student ("Not in allowed location").

5.7.1.5. Outcome

- If location and face verification are successful: Attendance is logged and confirmation is shown.
- If either step fails: An appropriate error message is returned.

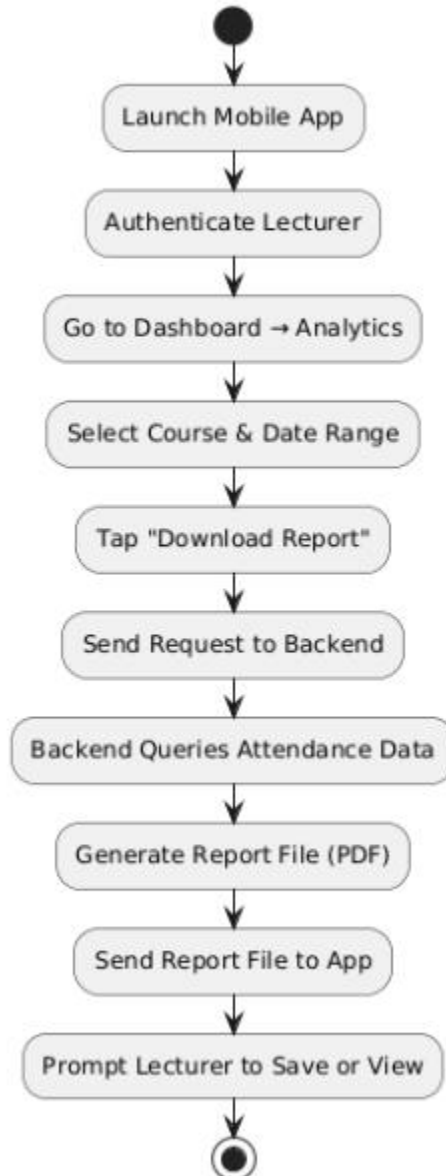
5.7.2. Download Attendance Report (Activity Diagram)

5.7.2.1. Purpose

To describe the sequential steps a lecturer follows in order to request, retrieve, and download an attendance report through the mobile app.

5.7.2.2. Diagram

Download Attendance Report Activity Diagram



5.7.2.3. Actors Involved

- **Lecturer** (User requesting the report)
- **Mobile App** (UI for navigating and making the request)
- **Backend Server** (Processes the request and formats the report)
- **Database** (Source of the attendance data)

5.7.2.4. Flow Description

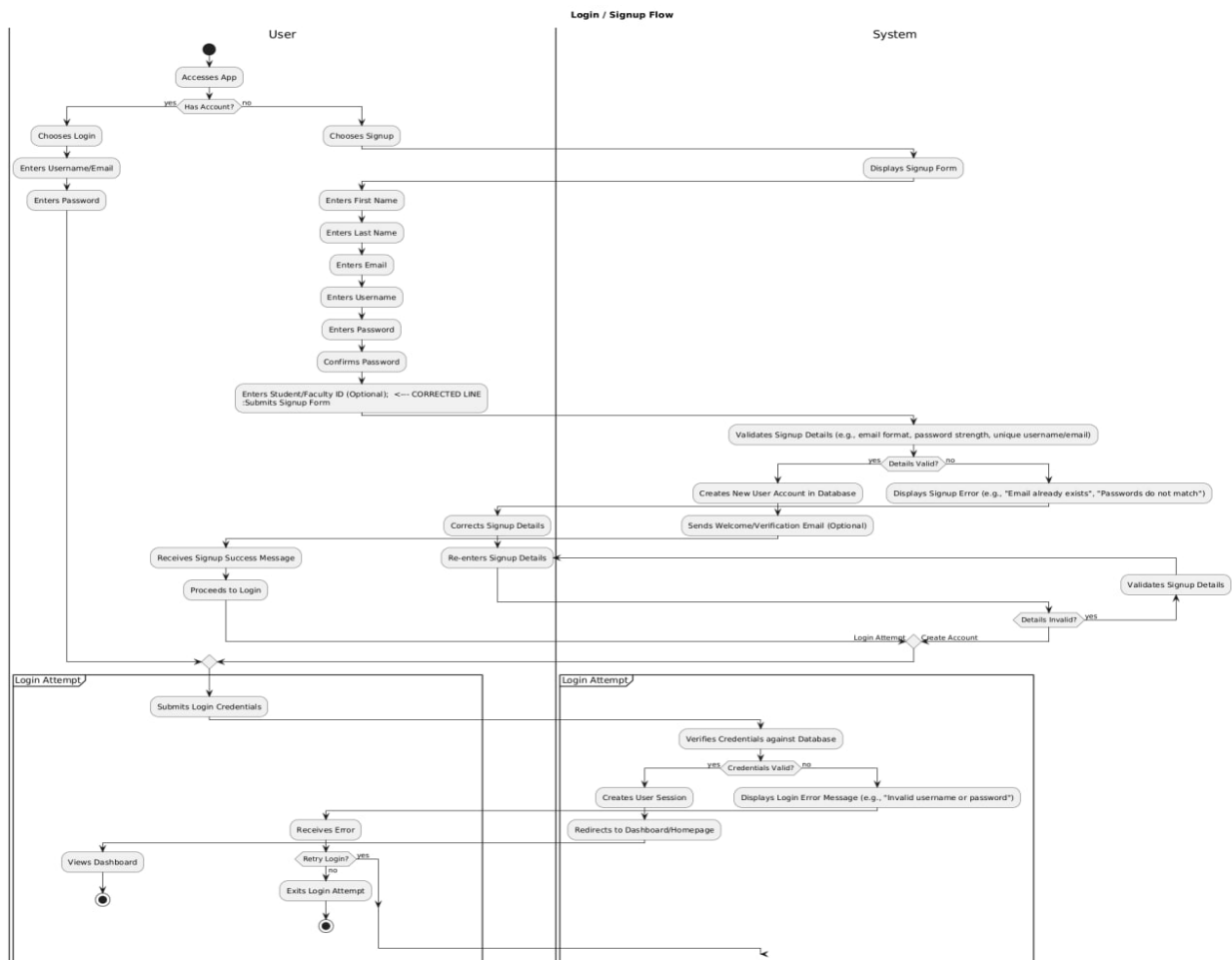
- i. **Start** – The process begins when the lecturer opens the mobile app.
- ii. **Authentication** – The lecturer logs in using their credentials.
- iii. **Dashboard Access** – The lecturer navigates to the Analytics section.
- iv. **Input Parameters** – The lecturer selects the relevant course and date range for the report.
- v. **Trigger Request** – The lecturer taps “Download Report”.
- vi. **Backend Request** – The mobile app sends this request to the backend server.
- vii. **Data Query** – The backend queries the database for matching attendance records.
- viii. **Report Generation** – A formatted report (usually PDF) is created by the backend.
- ix. **Delivery** – The report is sent back to the mobile app.
- x. **Download Prompt** – The lecturer is prompted to view or save the report.

5.7.2.5. Outcome

- A downloadable attendance report (PDF or JSON) is presented to the lecturer.
- If the report cannot be generated, an appropriate message is returned.

5.7.3. Login Flow

5.7.3.1. Diagram



Login / Signup Activity Diagram

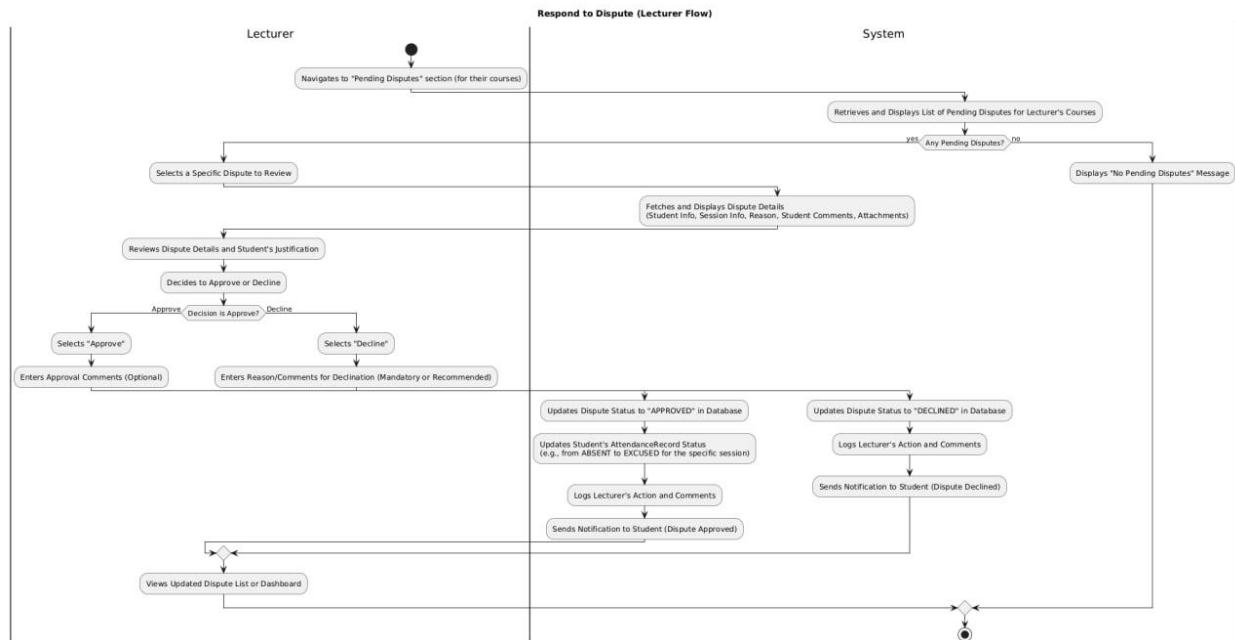
5.7.3.2. Brief Explanation of Flow

This diagram illustrates the initial user interaction with the application.

- i. The **User** accesses the app.
- ii. A decision point checks if the user **Has Account?**.
- iii. **If Yes (Login):** The user chooses to log in, enters their credentials. The **System** validates these. If valid, a session is created, and the user is directed to the dashboard. If invalid, an error is shown, and the user can retry.
- iv. **If No (Signup):** The user chooses to sign up. The **System** displays a signup form. The user fills in details (name, email, password, role, relevant ID). The **System** validates these details. If valid, a new account is created, and the user is typically prompted to log in. If invalid, errors are displayed for correction.

5.7.4. Respond to Dispute by Lecturer

5.7.4.1. Diagram



Respond to Dispute by Lecturer Activity Diagram

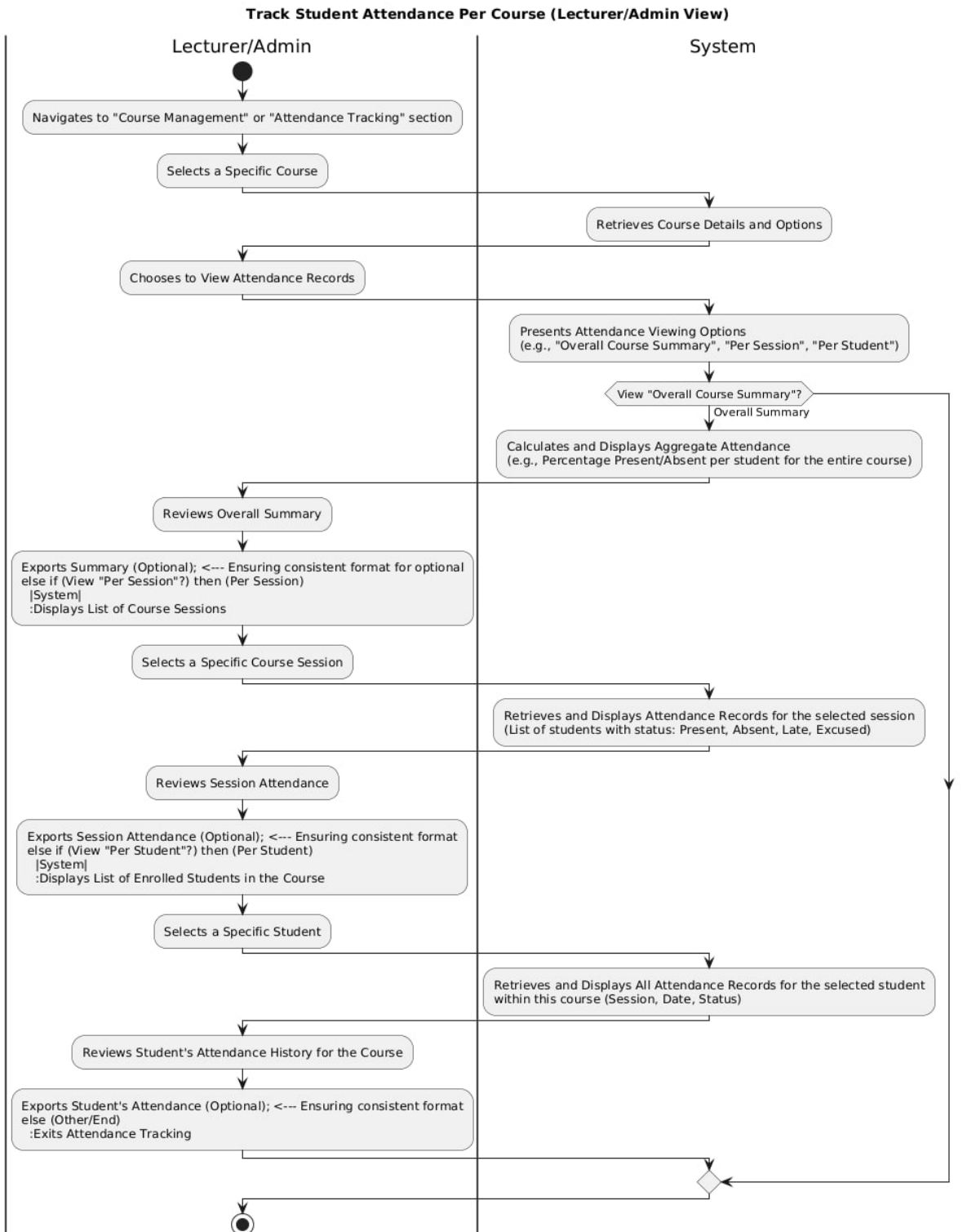
5.7.4.2. Brief Explanation of Flow

This diagram details the process a lecturer follows to handle an attendance dispute.

- i. The **Lecturer** logs in and navigates to the "Pending Disputes" section.
- ii. The **System** retrieves and displays the list of disputes relevant to the lecturer's courses.
- iii. If disputes exist, the **Lecturer** selects one to review.
- iv. The **System** fetches and displays the full details of the selected dispute.
- v. The **Lecturer** reviews the information and decides whether to **Approve** or **Decline**.
- vi. **If Approve:** The lecturer confirms, optionally adds comments. The **System** updates the dispute status to "APPROVED", updates the student's corresponding AttendanceRecord (e.g., to "EXCUSED"), logs the action, and notifies the student.
- vii. **If Decline:** The lecturer confirms, provides a reason/comments. The **System** updates the dispute status to "DECLINED", logs the action, and notifies the student.
- viii. The lecturer can then view the updated dispute list.

5.7.5. Track Student Attendance per Course

5.7.5.1. Diagram



Track Student Attendance per Course Activity Diagram

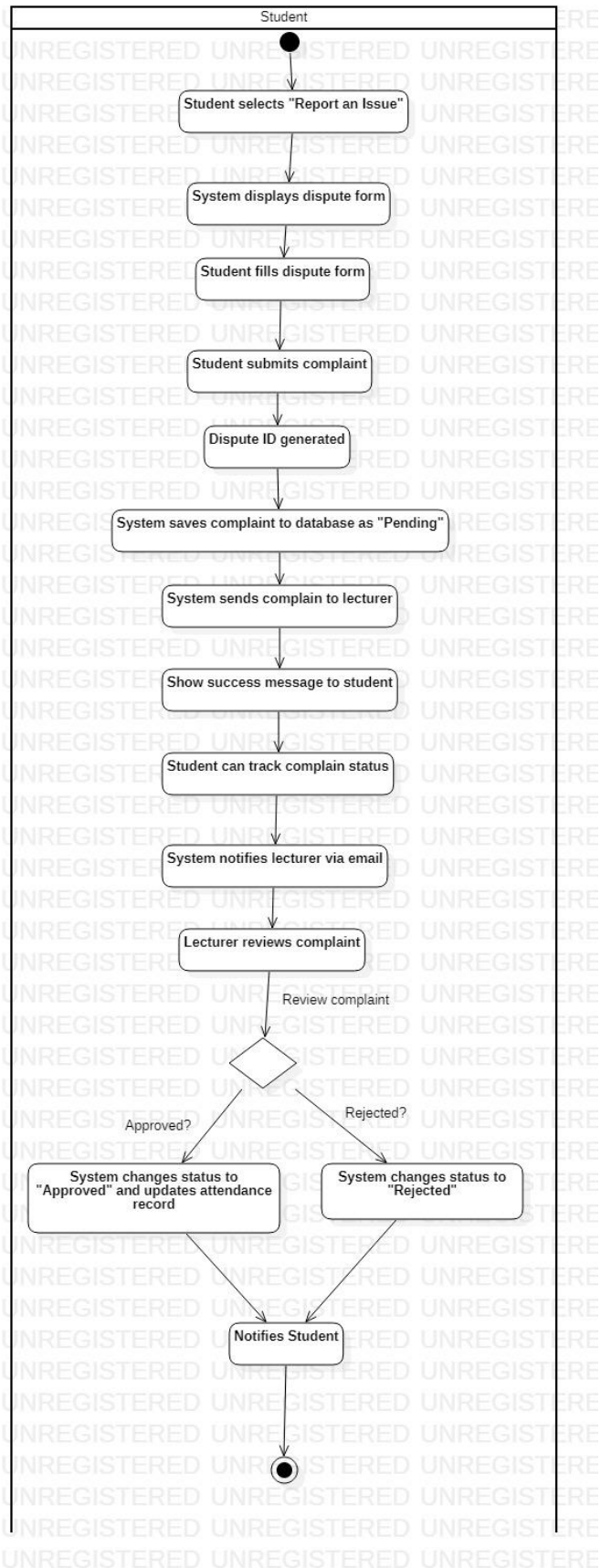
5.7.5.2. Brief Explanation of Flow

This diagram shows how a lecturer or admin can view attendance records for a specific course.

- i. The **Lecturer/Admin** logs in and selects a specific course to inspect.
- ii. The **System** presents options for viewing attendance (e.g., "Overall Course Summary," "Per Session Details," "Per Student History").
- iii. **If "Overall Course Summary":** The **System** calculates and displays aggregate attendance statistics for all students in the course.
- iv. **If "Per Session Details":** The **System** lists course sessions. The user selects a session, and the **System** displays attendance for all students in that session.
- v. **If "Per Student History":** The **System** lists enrolled students. The user selects a student, and the **System** displays their complete attendance record for that course.
- vi. The user can review the displayed information and optionally export it.

5.7.6. "Make a Complaint"

5.7.6.1. Diagram Flow



5.7.6.2. Key Steps

- **Complaint Submission:** Student fills a form (stored in Firestore).
- **Real-Time Alert:** Firebase triggers an email to the lecturer.
- **Resolution:** Lecturer updates attendance status.
- **Notification:** Student receives in-app/email confirmation.

6. Conclusion

The system design presented in this document provides a comprehensive and modular blueprint for building a modern, mobile-based attendance management system designed for academic institutions.

Each module has been carefully structured to ensure usability, scalability, and integration with mobile platforms using Firebase and real-time APIs. The inclusion of role-based access, automated logging, real-time validation, and leave/dispute management ensures the system meets both the functional and non-functional requirements outlined in the SRS.

The use of Unified Modeling Language (UML) diagrams has helped visualize system behavior, data flow, and user interactions. These design artifacts serve not only as implementation guides for developers but also as communication tools for stakeholders and project reviewers.

As the system progresses toward development and deployment, this design document will serve as a foundational reference. Future work may involve extending the system with biometric enhancements, and administrative controls for multiple institutions.

In summary, this document outlines a reliable, secure, and user-friendly design that ensures transparent and efficient attendance management, setting the stage for a robust digital transformation in academic environments.